Agent vocal temps réel
Bilan technique de l'architecture

Projet D1 (5A) – cascade STT → LLM → TTS, streaming de bout en bout. Accent mis sur la partie agentique : orchestration, appel d'outils, annulation, repli.

Document généré le 2026-10-23 à partir de la lecture du code source (`src/`) et des docs du dépôt. Les faits viennent du code, sauf mention H (hypothèse) ou R (recommandation).

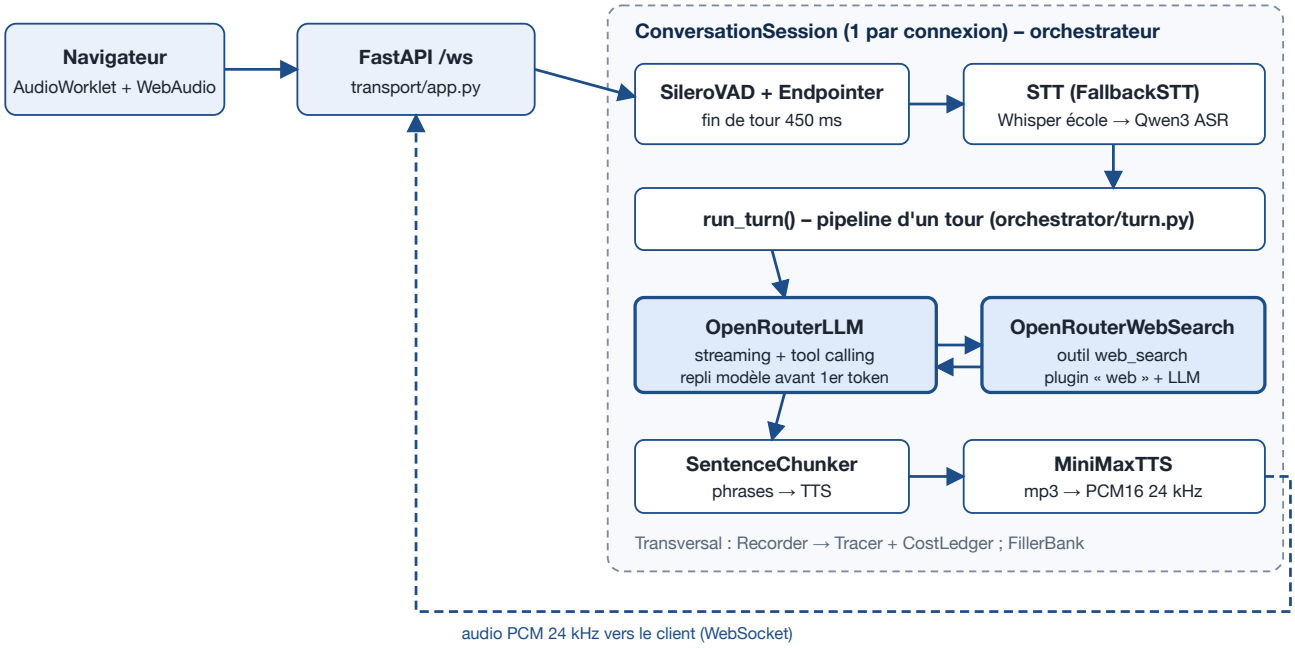
Réponse courte à « comment appelle-t-on les agents ? »

Il n'y a **pas de framework multi-agents** (ni LangChain, LangGraph, AutoGen, CrewAI, ni SDK d'agents) : le projet n'importe que `httpx`, `fastapi`, `numpy/onnx` et `miniaudio`. L'« agent » est le système complet, et il est piloté par du code asyncio écrit à la main :

1. À chaque connexion WebSocket, `ws_endpoint` appelle `build_providers()` (fabrique) qui instancie les briques **STT / LLM / TTS / WebSearch**, chacune derrière un `Protocol` Python.
2. Elles sont injectées dans **une** `ConversationSession` (machine à états *listening* → *thinking* → *speaking*).
3. La session lance chaque tour via `asyncio.create_task(self._turn(…))`, qui appelle `run_turn()`, lequel appelle `stt.transcribe()`, `llm.stream()` puis `tts.stream()`.
4. La seule vraie autonomie « agentique » est la **boucle d'appel d'outil** du LLM (`web_search`, au plus un appel par réponse), gérée dans `OpenRouterLLM._stream_one`.

1. Vue d'ensemble

Le système est un **agent conversationnel vocal duplex** : le navigateur capture le micro (AudioWorklet, PCM16 16 kHz) et l'envoie par WebSocket à un serveur FastAPI. Le serveur détecte la parole (VAD Silero), transcrit (STT), génère la réponse (LLM en flux, avec outil de recherche web), la synthétise (TTS par morceaux) et renvoie du PCM16 24 kHz. L'utilisateur peut couper la parole à l'agent (barge-in).



Pile technique (faits vérifiés dans le dépôt)

Couche	Technologie	Fichier
Client	AudioWorklet PCM16 16 kHz, trames de 512 échantillons, lecture WebAudio planifiée, `echoCancellation`	`transport/static/`
Transport	FastAPI + WebSocket `/ws`, binaire = PCM, texte = JSON de contrôle	`transport/app.py`
VAD	Silero (ONNX CPU) + `Endpointer` à hystérésis (0,5 / 0,35), pré-roll 300 ms, fin de tour 450 ms	`vad/`
STT	Whisper large-v3-turbo (serveur école, API compatible OpenAI) ; secours `qwen/qwen3-asr-1.7b` via OpenRouter	`stt/`, `fallback/`
LLM	OpenRouter `/chat/completions` en SSE ; principal DeepSeek V4.1 Flash, repli Qwen3.8 27B (DECISIONS 8) ; raisonnement désactivé	`llm/openrouter.py`
Recherche	OpenRouter, plugin `web` (3 résultats), résumé par le LLM	`search/`
TTS	`minimax/speech-2.8-turbo`, 2 voix françaises tirées au hasard par conversation, mp3 en flux décodé par `miniaudio`	`tts/`
Mesure	`Recorder` → `Tracer` JSONL (`time.monotonic_ns`) + `CostLedger`	`metrics/`

2. Comment les « agents » sont appelés

2.1 Contrats : des `Protocol` structurels

`src/interfaces.py` définit quatre contrats de typage structurel (duck typing vérifiable) : aucune classe de base à hériter, une brique est interchangeable dès qu'elle expose la bonne signature.

Protocol	Signature	Implémentations
`STT`	`async transcribe(pcm16, sample_rate) -> str`	`SchoolWhisper`, `FallbackSTT`, `FakeSTT`
`LLM`	`stream(messages) -> AsyncIterator[str | SearchNotice]`	`OpenRouterLLM`, `FakeLLM`
`WebSearch`	`async search(query) -> str`	`OpenRouterWebSearch`
`TTS`	`stream(text) -> AsyncIterator[bytes]`	`MiniMaxTTS`, `FakeTTS`

Point de conception notable : le flux du LLM est de type `str | SearchNotice`. Un **événement de contrôle** (`SearchNotice`) est intercalé dans le flux de texte, ce qui permet à l'orchestrateur de réagir (accroche vocale) sans couplage direct avec l'outil.

2.2 Fabrique et injection de dépendances

`factory.build_providers(settings, voice, client, on_event, dry_run)` assemble la chaîne et retourne `(stt, llm, tts)` :

- **dry-run** : `FakeSTT/FakeLLM/FakeTTS`, aucun réseau (tests, CI, `VOICE_DRY_RUN=1`).
- **réel** : `FallbackSTT(SchoolWhisper, SchoolWhisper[OpenRouter])` ; `OpenRouterWebSearch` uniquement si `search.enabled`, injecté dans `OpenRouterLLM(searcher=…)` ; `MiniMaxTTS` avec le débit propre à la voix.
- Un **unique** `httpx.AsyncClient` partagé (keep-alive, TLS déjà négocié) ; le callback `on_event` (le `Recorder`) est injecté dans chaque brique, ce qui centralise la trace et le coût.

2.3 Cycle de vie et portée

Portée	Objets	Où
Processus (lifespan FastAPI)	client HTTP, `CostLedger`, `SileroVAD` (partagé), banques d'accroches pré-synthétisées par voix, tâche `_keepalive` (toutes les 25 s : `warmup()` des 3 briques)	`app.lifespan`
Connexion WebSocket	`Tracer` / `Recorder` propres, voix tirée au hasard, nouveaux providers, `ConversationSession`, historique	`app.ws_endpoint`
Tour de parole	tâche `_turn`, tâche STT spéculative, tâches de synthèse par morceau, tâche *drainer*	`session._turn`, `turn.run_turn`

2.4 Chaîne d'appel d'un tour

```
ws_endpoint
 └─ session.on_mic(pcm)                  # VAD à chaque trame de 32 ms
     └─ Endpointer.push → speech_end
         └─ _start_turn()                # asyncio.create_task
             └─ _turn()                  # états + accroches + gestion d'erreur
                 └─ run_turn()           # pipeline pur, callbacks on_user_text / on_chunk /
on_audio / on_search
                     ├─ stt.transcribe(audio)         (ou tâche spéculative réutilisée)
                     ├─ llm.stream([system]+history)  → deltas str | SearchNotice
                     │    └─ (si outil) searcher.search(query) → 2e requête LLM
                     ├─ SentenceChunker.feed(delta)   → morceaux
                     └─ tts.stream(morceau)           → PCM, tâches parallèles, sortie ordonnée
```

L'orchestration est donc **impérative et événementielle** : `run_turn` est la fonction de flux, la `ConversationSession` fournit les callbacks (envoi JSON/audio, changements d'état, accroches). `run_turn` ne connaît ni le réseau client ni l'état de session, ce qui le rend testable seul (`tests/test_turn_dry_run.py`).

3. Partie agentique en détail

3.1 Boucle d'appel d'outil (function calling)

Dans `OpenRouterLLM._stream_one`, le modèle reçoit l'outil `web_search(query: string)` au format OpenAI `tools`. C'est une boucle **ReAct bornée à une itération** :

#	Étape	Détail technique
1	Requête 1 en flux avec `tools=[WEB_SEARCH_TOOL]`	SSE ligne à ligne ; `reasoning.enabled=false` ; `provider.sort=latency` ; `include_usage`
2	Accumulation des `tool_calls`	Deltas fusionnés par `index` dans `calls{idx: {id, name, args}}` (arguments JSON reçus par fragments)
3	Décision	Si `calls` vide → réponse directe. Sinon : requêtes parallèles **regroupées en une** (`" ; "` .join) ; repli sur le dernier message utilisateur si les arguments sont illisibles
4	Événement de contrôle	`yield SearchNotice(query)` → l'orchestrateur joue l'accroche « je regarde ça »
5	Exécution de l'outil	`await searcher.search(query)` ; sur `SearchError` → résultat `SEARCH UNAVAILABLE` (le LLM doit dire qu'il n'a pas pu vérifier, sans inventer)
6	Requête 2 **sans outil**	Messages + `assistant.tool_calls` + `tool` (résultat) ; `use_tools=False` garantit l'arrêt de la boucle
7	Flux de réponse	Deltas texte → chunker → TTS, comme une réponse normale

> **L'outil est lui-même un appel LLM (« outil-agent »)**
>
> `OpenRouterWebSearch.search` envoie une requête non streamée à `/chat/completions` avec le plugin `{"id":"web","max_results":3}` et un prompt système de « outil de recherche » (2 à 4 phrases, sans markdown ni URL, température 0,2). Le texte renvoyé est donc une **synthèse déjà rédigée**, pas des résultats bruts : le contexte du LLM principal reste court. Mesuré (DECISIONS 13b) : décision d'outil ≈ 0,5–1 s, recherche ≈ 2–3,4 s, ≈ 0,007 $ par recherche.

3.2 Robustesse face aux appels d'outil mal formés : `ToolMarkupGuard`

Défaut observé en test réel : sur une question à deux sujets, DeepSeek écrit l'appel d'outil **en texte** (balises `<｜DSML｜…>`) dans `content` au lieu de `tool_calls`, et le TTS le lisait à voix haute. Le garde est un filtre à état sur le flux :

- `feed(delta)` retient tout ce qui suit le motif `DSML` (regex) et conserve un tampon si le delta se termine par un début de balise `<` (moins de 12 caractères), car la balise peut être coupée entre deux deltas.
- En fin de flux, si `guard.active` et qu'aucun vrai `tool_call` n'existe, les requêtes sont extraites (regex `name="query"`) et un appel synthétique `call-markup` est créé : le flux se poursuit comme un appel normal.

3.3 Résilience en couches

Niveau	Mécanisme	Limite assumée
LLM	Liste `[modèle, modèle_alt]` ; passage au suivant sur `LLMError` (HTTP ≠ 200, **erreur dans le flux malgré HTTP 200**, détectée)	Seulement **avant le premier delta** (`started`) : reprise propre impossible après
Recherche	`SearchError` → `SEARCH UNAVAILABLE` , timeout 20 s	Silence de 2–3 s après l'accroche
STT	`FallbackSTT` : 2e fournisseur sur toute exception ; STT spéculatif retombe sur un appel normal s'il échoue	Pics de 33–35 s observés sur les deux (DECISIONS 3)
Session	`turn_error` : message utilisateur retiré de l'historique, retour à l'écoute, message d'erreur au client	—
Mode dégradé	Après `stt.fallback_to_text_after` =2 échecs STT consécutifs : `text_only` (micro ignoré), message `fallback: text` , réarmement par `voice_retry`	Vérifié par test, pas en panne réelle (ROADMAP 6)

3.4 Machine à états et concurrence

États : `LISTENING → THINKING → SPEAKING → LISTENING` . `agent_active` = THINKING ou SPEAKING. L'état est poussé au client à chaque transition (`state`).

Tâches `asyncio` concurrentes dans une session :

Tâche	Rôle	Annulation
`_task` (`_turn`)	STT → LLM → TTS d'un tour	par barge-in ou fermeture
`_spec`	STT spéculatif lancé après `speculative_ms` =200 ms de silence ; résultat réutilisé via `stt_task`	si la parole reprend, ou si l'audio du tour diffère (reprise `_carry`)
`_failsafe`	Passe en écoute si le client ne confirme jamais `playback_ended` (durée jouée + 5 s)	au début de tour
drainer + `synth` ×N (dans `run_turn`)	Synthèse parallèle des morceaux ; une `Queue` par morceau, drainées **dans l'ordre** via une file de files (`ordered`)	`except BaseException` annule tout

Chevauchement : le chunker coupe le flux du LLM sur la première ponctuation (`first_min` =8 caractères, virgule admise), puis sur fin de phrase (`rest_min` =20, `tts.min_chunk_chars`). Le TTS démarre donc avant la fin de la génération ; l'ordre de lecture est garanti malgré la synthèse parallèle.

3.5 Barge-in : annulation coopérative

Le VAD tourne en permanence. Tant que l'agent est actif, le seuil est relevé (`barge_in_min_prob` =0,75, anti-écho) et `barge_in_min_speech_ms` =300 ms de parole voisée sont requis. Séquence dans `_barge_in()` :

1. Trace `barge_in_detected` .
2. `task.cancel()` sur `_task` et `_failsafe` , puis `await gather(…, return_exceptions=True)` : l'annulation se propage à `run_turn` → tâches TTS → fermeture des générateurs → coupure des requêtes HTTP (LLM et TTS).
3. Si la phrase de l'utilisateur n'avait pas encore été transcrite, son audio est mis de côté (`_carry`) et **repris avec la suivante**.

4. **Troncature de l'historique** : durée jouée estimée = temps depuis le premier audio ; `heard_text()` reconstitue le texte entendu en coupant le morceau en cours proportionnellement, à une frontière de mot ; ajout de « [interrompu par l'utilisateur] » (`history_content`). Si rien n'a été entendu, le message assistant est supprimé.
5. Message `interrupt` au client (arrêt de lecture) ; le client répond `interrupted(played_s)` et `on_interrupted` **affine** la troncature avec la durée réellement jouée (`playback_stopped` tracé avec `ack_ms`).

3.6 Accroches (fillers) : masquer l'attente

- **Accroche générique** (« Hmm, voyons. ») : choisie dès la fin de tour si l'énoncé dure ≥ 1,8 s, avant même le STT. 12 textes pré-synthétisés par voix, mis en cache sur disque (`data/fillers/`), silences rognés. `FillerBank.pick()` choisit parmi les 3 durées les plus proches de l'attente moyenne (EMA) et jamais deux fois la même.
- **Accroche de recherche** (« Je regarde ça. ») : déclenchée par `SearchNotice`, seulement si aucun morceau de réponse n'est parti (`len(self._texts) > self._off` évite de décaler les rangs).
- **Cohérence du prompt** : `FILLER_HINT` est ajouté au prompt système (« tu viens de dire … : enchaîne directement »), et l'historique stocke *accroches + réponse*.
- **Métriques honnêtes** : « premier son » (accroche) et « réponse » (contenu, `content_ms`) sont deux latences distinctes ; l'attente d'une recherche n'alimente pas l'EMA.

3.7 Contexte, prompt et mémoire

- Prompt système composé par `build_system_prompt` : consigne de base + date du jour + `search_hint` si la recherche est active.
- Mémoire = liste `history` en RAM, bornée à `max_history_messages`=20 (`del history[:-max]`). Elle ne survit pas à la connexion. Le résultat brut de recherche n'y est **pas** conservé (seulement la réponse).
- Aucune mémoire long terme, aucun RAG, aucune planification multi-étapes.

4. Structure du dépôt

Module	Contenu	Rôle
`src/interfaces.py`	`4 Protocol` + `SearchNotice`	Contrats
`src/factory.py`, `config.py`	assemblage, `Settings` (.env + YAML)	DI, configuration
`src/orchestrator/`	`session.py` (338 l.), `turn.py`, `chunker.py`, `truncation.py`	Cerveau : états, tour, découpe, troncature
`src/llm/`, `search/`	`OpenRouterLLM`, `ToolMarkupGuard`, `OpenRouterWebSearch`	Agent LLM + outil
`src/stt/`, `fallback/`, `tts/`	Whisper, `FallbackSTT`, MiniMax, `FillerBank`, voix	Fournisseurs audio
`src/vad/`	`SileroVAD`, `Endpointer` (logique pure), `scan.py`	Détection de parole
`src/audio/`	décodage mp3 progressif, PCM, bruit à SNR fixé	Traitement signal
`src/metrics/`	`Recorder`, `Tracer`, `CostLedger`, analyse	Observabilité
`src/replay/`	micro simulé temps réel, client WS, essais	Campagne reproductible
`src/transport/`	FastAPI + `static/index.html`, `worklet.js`	E/S
`scripts/`, `tests/`, `notebooks/`	campagne, smoke Chrome, 18 fichiers de tests, analyse p50/p95	Qualité et mesure

Protocole WebSocket

Sens	Messages
Client → serveur	binaire PCM16 16 kHz ; JSON `text`, `playback_ended`, `interrupted{played_s}`, `voice_retry`
Serveur → client	binaire PCM16 24 kHz ; JSON `ready`, `state`, `vad`, `transcript`, `user_pending`, `agent_chunk`, `searching`, `metrics`, `audio_end`, `interrupt`, `truncate`, `fallback`, `error`

5. Observabilité et mesure

- **Un seul point d'entrée** : chaque brique reçoit `on_event` = `Recorder` et appelle `rec("tts_request", …)`. Le *stage* est le préfixe du nom (`tts_request` → `tts`). L'événement spécial `usage` alimente le `CostLedger` au lieu des traces.
- **Horloges** : durées en `time.monotonic_ns` (`Tracer.now_ms`), horodatage mur UTC dans chaque ligne JSONL.
- **Métriques** par tour (message `metrics`) : `stt_ms`, `llm_ttft_ms`, `tts_ttfb_ms`, `first_sound_ms`, `content_ms`, `search_ms`, `gap_ms`.
- **Campagne** (`scripts/run_campaign.py`, `configs/campaign.yaml`) : phases *stt*, *vad*, *latency*, *barge*, voix et bruit synthétiques (blanc, foule) à SNR ∈ {20,10,5,0} dB.

Résultats mesurés (run1, DECISIONS 11 ; petits échantillons)

Indicateur	Valeur
Latence sans accroche (fin de parole → 1er son), p50 / p95	2,06 s / 2,45 s
Avec accroche : 1er son / début de réponse	0,52 s / ≈ 2,1 s ; 22 % de réponses prématurées
WER Whisper	≈ 0 % jusqu'à 5 dB ; blanc 0 dB 16 % ; foule 0 dB 67 %
Phrases coupées par la fin de tour	20 % à 450 ms, 10 % à 600 ms, 2 % à 800 ms
Interruptions correctement gérées	14/21 (67 %), arrêt propre 21/21
Tour avec recherche : contenu après fin de parole	≈ 5,5–6,6 s (DECISIONS 13b)
Coût cumulé	≈ 2,2 $ au 2026-10-22 (budget 15 €)

6. Limites et pistes (analyse de lecture du code)

Points d'attention H – à confirmer par test avant de les considérer comme des défauts

- **Mono-utilisateur** : `SileroVAD` est partagé entre connexions (état interne), documenté comme limite.
- **Pas de reprise LLM après le premier delta** : une erreur en cours de flux fait échouer le tour.
- **Une recherche par réponse** : pas de boucle multi-outils ; les requêtes multiples sont fusionnées en une chaîne.
- **Mémoire** : 20 messages, perdus à la reconnexion ; la valeur 5 proposée par l'étudiant reste à comparer (ROADMAP).
- **Troncature** : estimation proportionnelle (écart ≤ ~0,4 s), les silences d'attente comptent dans la durée jouée.
- **Réglages recommandés non appliqués** : fin de tour 600 ms, seuil barge-in 0,5–0,6 (DECISIONS 11).
- **Écho** : casque supposé ; pas d'AEC serveur.

Extensions possibles R

- Second outil (calendrier, calcul) : l'architecture le permettrait en généralisant `WEB_SEARCH_TOOL` en registre d'outils et `WebSearch` en `Tool` ; la boucle devrait alors passer de « une itération » à « N itérations bornées ».
- Recherche spéculative lancée plus tôt, seconde accroche si la recherche dépasse un seuil (ROADMAP « Attente de recherche »).
- Routage entre fournisseurs (étape 10) : `FallbackSTT` montre déjà le patron, à généraliser en `FallbackLLM/TTS` avec un champ `route` dans les traces.

Sources : `src/interfaces.py`, `factory.py`, `orchestrator/session.py`, `orchestrator/turn.py`, `llm/openrouter.py`, `search/openrouter_web.py`, `fallback/stt.py`, `vad/endpointer.py`, `transport/app.py`, `configs/base.yaml`, `docs/*.md`.